

作业-打造简易版“贾维斯”边缘智能

目的：通过设备端边缘智能开发，掌握物联网应用平台的端边管云方法。

描述：小组作业，3-5 人一组，以智慧小灯为业务参考，可自行剪裁或增加。

提交时间：第 16 周，第 17 周系统演示，答辩

平台：开发板为 esp32s3box-lite，搭载 ESP32-S3 AI SoC，在芯片内置的 512 KB SRAM 之外，还集成了 16 MB QSPI flash 和 8 MB Octal PSRAM。拥有离、在线语音功能，有 3 个自定义的功能按键。基于 espidf 进行开发编译。



作业说明：在钢铁侠电影中，贾维斯作为智能助手，以其自然流畅的语音交互、实时环境感知和智能决策能力而广受喜爱。近年来，随着智能家居技术的发展，一些智能家具已经具备智能感知和自动控制功能。例如，空调可以通过红外传感器或 GPS 信息自动检测室内是否有人，并进行智能开关控制。随着大模型技术的进步，智能语音聊天系统逐步进入嵌入式设备，进一步推动了智能家居的广泛应用。用户不仅可以通过语音控制单个设备，还可以通过大模型实现复杂的场景联动，覆盖多设备的智能操作，并可以对大模型进行引导操作，如：当用户说打开空调，并在空气干燥时打开加湿器，大模型便会同时设定三条规则，打开空调，检测空气湿度是否干燥，打开加湿器。示例中实现了一个支持语音控制、定时开关的智能小灯。具体流程如下：

1. **【采】：**当设备被“你好，小智”唤醒后，则会采集音频数据发送给远端。
2. **【析】：**大模型接收到音频数据后，自动提取用户意图，生成相应的控制指令，并将反馈音频返回设备端，实现即时交互。
3. **【管】：**设备端根据大模型返回的控制指令，启动定时器操作，定时结束后与 HA 平台进行交互。
4. **【云】：**各类物联网设备通过不同协议接入 HA 云平台，实现统一的设备管理和状态同步，支持多种智能家居场景的联动控制。

作业要求：

小组规模：3-5 人

任务内容：实现上述智能家居处理流程，包括指令生成、设备控制和云端管理，并录制功能演示视频。

设备支持：以小组为单位领取一块 esp32s3box-lite 开发板，并购买智能家居设备，总预算不超过 500 元，可购买多件设备报销。

文档要求：撰写详细的技术文档，包括但不限于系统功能，技术方案。文档中需标明各组成员的姓名、学号、具体分工和贡献比例。

提交方式：将源代码、演示视频和说明文档打包提交至 Canvas 平台。

文件命名格式：组长姓名-组长学号。

示例代码下载地址：交大云盘 作业-边缘智能

链接: <https://pan.sjtu.edu.cn/web/share/ac7f6165c80ff6b7e860e9e8b2210be1>, 提取码: fkrq

示例：智能小灯控制

一. Home Assistant 部署

1.1 基于 docker 的安装方法

Home Assistant (HA) 是一个开源的家庭自动化平台，旨在将各种智能家居设备和服务集成在一起，实现全面的自动化控制和智能场景管理。HA 作为一个集成平台，通过抽象和标准化不同协议的物联网设备，提供统一的控制接口，使用户能够在统一的系统中管理和自动化各种智能设备。

1.1.1 安装 Docker

```
curl -fsSL https://get.docker.com | sh
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

1.1.2 运行 HA 容器

```
docker pull ghcr.io/home-assistant/home-assistant:stable
```

```
docker run -d \
```

```
    --name homeassistant \
```

```
    --restart=unless-stopped \
```

```
    -e TZ=Asia/Shanghai \
```

```
    -v /home/username/homeassistant:/config \
```

```
    --network=host \
```

```
    ghcr.io/home-assistant/home-assistant:stable
```

通过服务器 ip:8123 端口在浏览器中进行访问

1.2 米家集成

```
cd /home/username/homeassistant
```

```
git clone https://github.com/XiaoMi/ha_xiaomi_home.git
```

```
cd ha_xiaomi_home
```

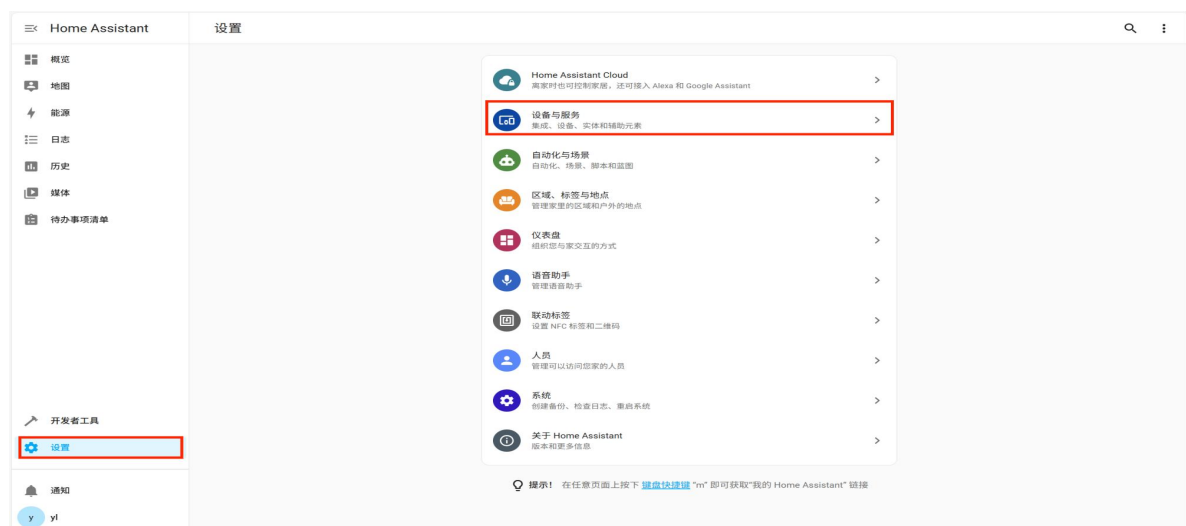
```
./install.sh ../
```

重启 HA 容器

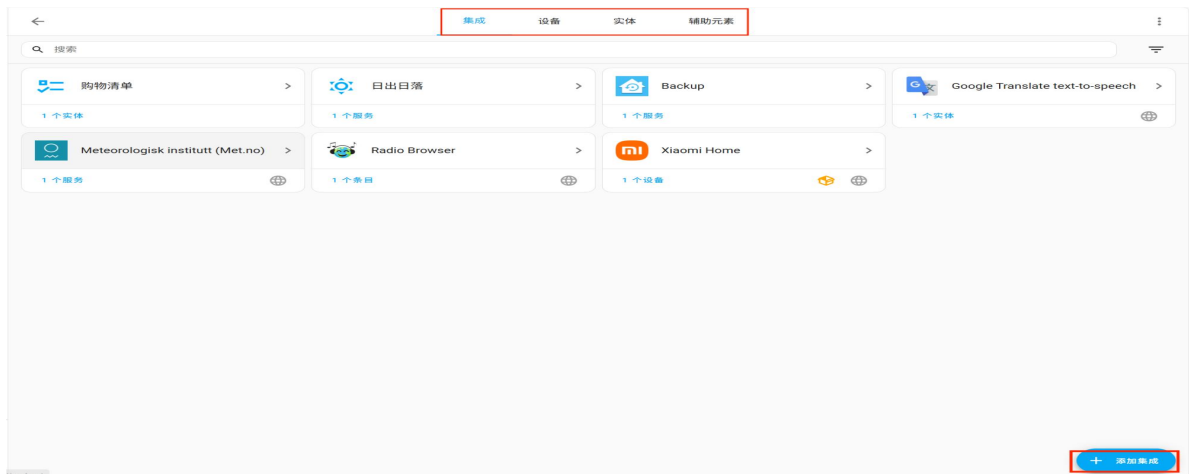
1.3 添加设备

登录 ip:8123，第一次登录需要注册

在左侧边栏，设置中选择设备与服务

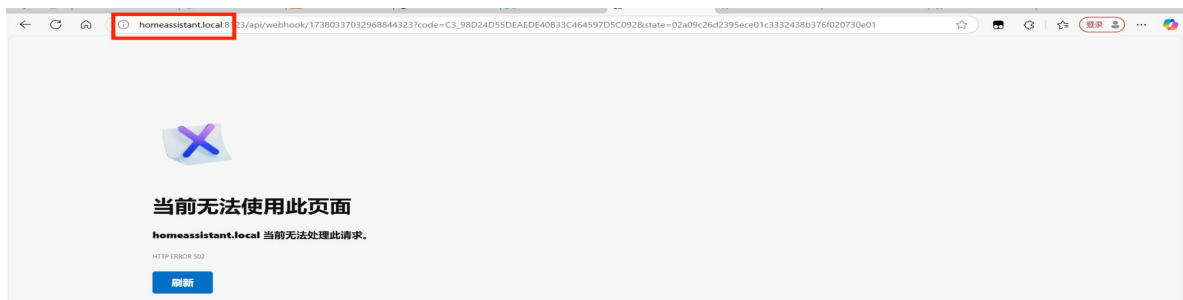


在上侧边栏中选择设备



右下角点击添加设备，搜索 xiaomi home 进行设备添加。

注：在登录小米账号时，手动将 homeassistant.local 改为服务器 ip，即可完成登录



1.4 token，设备 id，api 获取

1.4.1 token 获取

点击左下角用户头像，上侧边栏选择安全，点击长期访问令牌，创建令牌，可获取 token。token 只会显示一次，要注意保存。

二. espadf 安装

2.1 python 环境准备

espadf 需要 python3.8 以上环境，使用 `python --version` 输出 `python 3.8.0` 以上即可

2.2 espadf 下载

`git clone https://github.com/espressif/esp-adf.git`

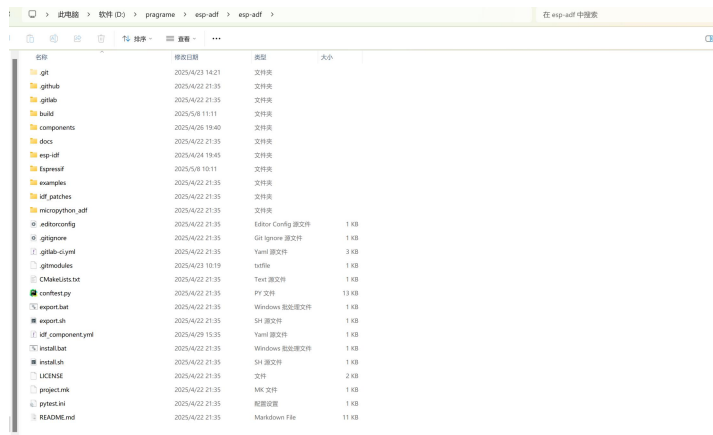
`git submodule update --init --recursive`

注：因为 github 网速较慢，下载可能会失败，如果经常下载失败可以尝试查看日志输出，单独在 github 上使用 zip 下载，解压到对应目录。可以在 `.gitmodules` 查看网址以及位置。

```
grame > esp-adf > esp-adf > .gitmodules
[submodule "esp-idf"]
    path = esp-idf
    url = https://github.com/espressif/esp-idf.git
[submodule "components/esp-adf-libs"]
    path = components/esp-adf-libs
    url = https://github.com/espressif/esp-adf-libs
[submodule "components/esp-sr"]
    path = components/esp-sr
    url = https://github.com/espressif/esp-sr.git
```

2.3 espadf 安装

在 esp-adf 根目录下执行



install.bat

export.bat

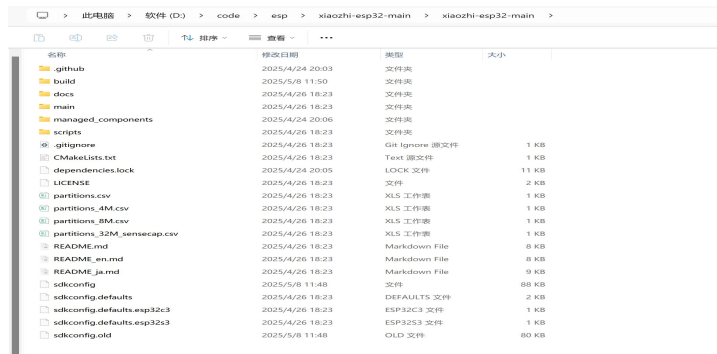
可使当前窗口有 idf 环境，在下次重新打开 cmd 窗口时需要重新执行。

或者将 esp-idf 和 esp-idf\tools 路径加入系统环境中。

三. 运行前设置

3.1 设置为 esp32S3

在 xiaozhi-esp32-main 主目录下



执行 idf.py set-target esp32s3

3.2 固件配置及软件烧录

idf.py menuconfig

Xiaozhi Assistant -> Board Type -> esp box lite

Serial flasher config -> Flash size -> 16MB

Partition Table -> Custom partition CSV file -> partitions.csv

Component config -> ESP PSRAM -> SPI RAM config -> Mode (QUAD/OCT) ->

Octal Mode PSRAM

idf.py flash monitor（后续代码更新只需要执行这一步）

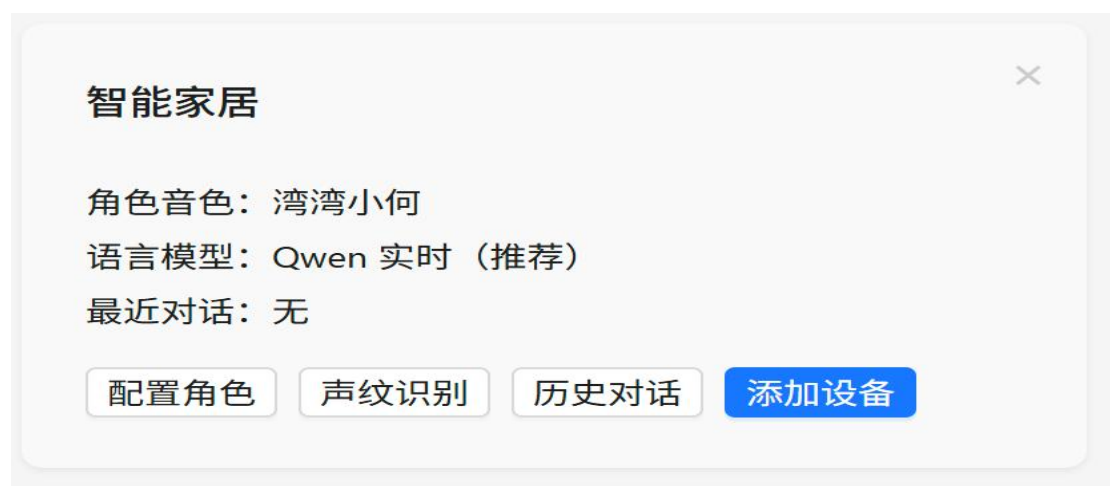
3.3 联网配置

手机 wifi 连接 Xiaozhi-xxx 热点，进行 wifi 配置。

四. 后端平台搭建

4.1 第三方平台 [xhttps://xiaozhi.me](https://xiaozhi.me)

注册账号后，点击控制台，添加设备，输入开发板的 6 位验证码，就可以添加设备。



点击配置角色，可以修改大模型一些提示词或者修改语言模型。建议使用 Doubao 1.5 Pro。

4.2 平台搭建（可选）

如果本地搭建平台，并直接与 HA 通信，可实现更简单和功能更丰富的物联网控制，并且可以选择更多的模型。

项目地址 <https://github.com/xinnan-tech/xiaozhi-esp32-server>

	.github	2025/5/11 11:33	文件夹	
	docs	2025/5/11 11:33	文件夹	
	main	2025/5/11 11:33	文件夹	
	.dockerignore	2025/5/11 11:33	DOCKERIGNORE 文...	1 KB
	.gitignore	2025/5/11 11:33	Git Ignore 源文件	3 KB
	Dockerfile-server	2025/5/11 11:33	文件	1 KB
	Dockerfile-web	2025/5/11 11:33	文件	1 KB
	docker-setup.sh	2025/5/11 11:33	SH 源文件	4 KB
	LICENSE	2025/5/11 11:33	文件	2 KB
	README.md	2025/5/11 11:33	Markdown File	14 KB
	README_en.md	2025/5/11 11:33	Markdown File	14 KB

在 docs/Deployment_all.md 中有详细的部署步骤。

在 xiaozhi--server 部署中可能会遇到环境问题，可以参考这篇帖子[解决 Exception: Could not find Opus library. Make sure it is installed.](#)-CSDN 博客

在 docs/homeassistant-integration.md 中有详细的与 HA 通信步骤。

五. 物联网模块

5.1 注册设备

在 main/application.cc 文件中的，Application::Start()函数，protocol_ -> OnAudioChannelOpened 中使用 thing_manager.AddThing(iot::CreateThing ("Name"));添加设备。

```

protocol_->OnAudioChannelOpened([this, codec, &board]() {
    board.SetPowerSaveMode(false);
    if (protocol_->server_sample_rate() != codec->output_sample_rate()) {
        ESP_LOGW(TAG, "Server sample rate %d does not match device output sample rate %d, resampling may cause distortion",
            protocol_->server_sample_rate(), codec->output_sample_rate());
    }
    SetDecodeSampleRate(protocol_->server_sample_rate(), protocol_->server_frame_duration());

    auto& thing_manager = iot::ThingManager::GetInstance();
    thing_manager.AddThing(iot::CreateThing("Lamp"));
    protocol_->SendIotDescriptors(thing_manager.GetDescriptorsJson());
    std::string states;
    if (thing_manager.GetStatesJson(states, false)) {
        protocol_->SendIotStates(states);
    }
});

```

5.2 设备代码

在 main/iot/lamp.cc 文件中，定义了一个物联网小灯。

```

Lamp::Lamp() : Thing("Lamp", "卧室里灯") {
    properties_.AddBooleanProperty("power", "灯是否打开", [this]() -> bool {
        return power_;
    });
    properties_.AddBooleanProperty("timer", "定时器是否启动", [this]() -> bool {
        return timer_;
    });
    methods_.AddMethod("TurnOn", "打开灯,当你听到类似10秒钟后,10分钟后开灯选择此方法,你需要将分钟,小时转化为秒后,当参数传递给此方法", ParameterList({
        Parameter("brightness", "亮度, 0-100, 默认为100", kValueTypeNumber, false),
        Parameter("timer", "计时器, 多少秒后开灯, 需要将分钟, 小时转化为秒, 默认为0", kValueTypeNumber, false)
    }), [this](const ParameterList& parameters) {
        TurnOnLamp(parameters); // 直接调用 TurnOnLamp
    });
    methods_.AddMethod("TurnOff", "关闭灯", ParameterList({
        Parameter("timer", "计时器, 多少秒后关灯, 需要将分钟, 小时转化为秒, 默认为0", kValueTypeNumber, false)
    }), [this](const ParameterList& parameters) {
        TurnOffLamp(parameters); // 直接调用 TurnOffLamp
    });
    methods_.AddMethod("CancelTimer", "取消定时器", ParameterList(), [this](const ParameterList&) {
        CancelTimer();
    });
    Application::GetInstance().UpdateIotStates();
};
}

```

此处为属性，类似一个大模型可以看到的变量

此处为方法，包括方法名称和描述，本地设备上需要执行的函数，以及函数参数

此处为设备描述模块，可以增加新的属性和方法。上图当说出开灯指令后，大模型则会返回“TurnOn”这个名称，本地则执行 TurnOnLamp 这个函数。

```

void Lamp::TurnOnLamp(const ParameterList& parameters) {
    turn_on = true;
    timer_ = true;
    int brightness = parameters["brightness"].number(); // 获取亮度
    int timer = parameters["timer"].number(); // 获取定时器时间 (秒)

    // 如果timer大于0, 启动定时器任务
    if (timer > 0) {
        StartTimerTask(timer, brightness); // 启动定时器任务, 并传递亮度参数
    } else {
        // 如果没有定时器, 立即执行开灯操作
        SendTurnOnCommand(brightness);
    }

    Application::GetInstance().UpdateIotStates();
}

```

上图会在本地启动定时器, 当定时器为 0 时或没有执行定时操作即定时时间参数为 0 时, 则执行 SendTurnOnCommand, 向 HomeAssistant 发送开灯指令。

```

void Lamp::SendTurnOnCommand(int brightness_) {
    const char* url = "http://10.119.13.189:8123/api/services/light/turn_on";

    std::string body = std::string("{\"entity_id\":\"" + entity_id + "\", \"brightness_pct\":" + std::to_string(brightness_) + "}");

    esp_http_client_config_t config = {};
    config.url = url;
    config.method = HTTP_METHOD_POST;
    config.timeout_ms = 5000;

    esp_http_client_handle_t client = esp_http_client_init(&config);

    std::string auth_header = "Bearer " + std::string(token);
    esp_http_client_set_header(client, "Authorization", auth_header.c_str());
    esp_http_client_set_header(client, "Content-Type", "application/json");

    esp_http_client_set_post_field(client, body.c_str(), body.length());

    esp_err_t err = esp_http_client_perform(client);

    if (err == ESP_OK) {
        ESP_LOGI(TAG, "TurnOn HTTP POST succeeded, status = %d", esp_http_client_get_status_code(client));
    } else {
        ESP_LOGE(TAG, "TurnOn HTTP POST failed: %s", esp_err_to_name(err));
    }

    esp_http_client_cleanup(client);
}

```

开发板通过 http 与 HA 通信, 发送 entity_id (ha 为小灯分配的 id) 和 brightness_pct (亮度参数) 给 HA, HA 执行开灯方法。

5.3 注册新设备

- 1.可以在 `main/iot` 中创建新代码，继承 `thing` 基类

2. 定义属性

3. 定义方法

4. 定义设备名称

在代码最后加入，即可定义设备名称 `DECLARE_THING(Name);`

5. 注册设备（参考 4.1）